

Thonny-uPyPi 包管理插件需求规格说明书

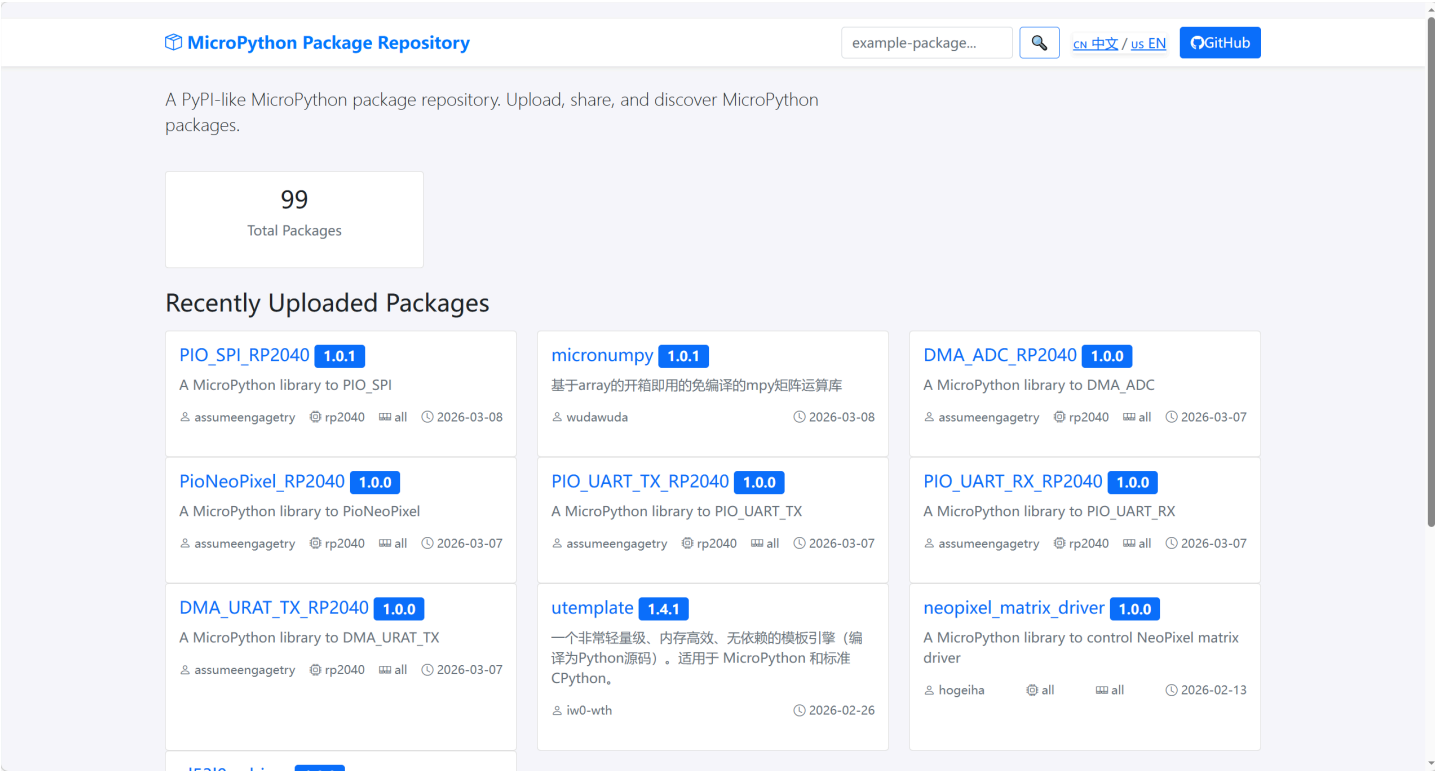
一、文档概述

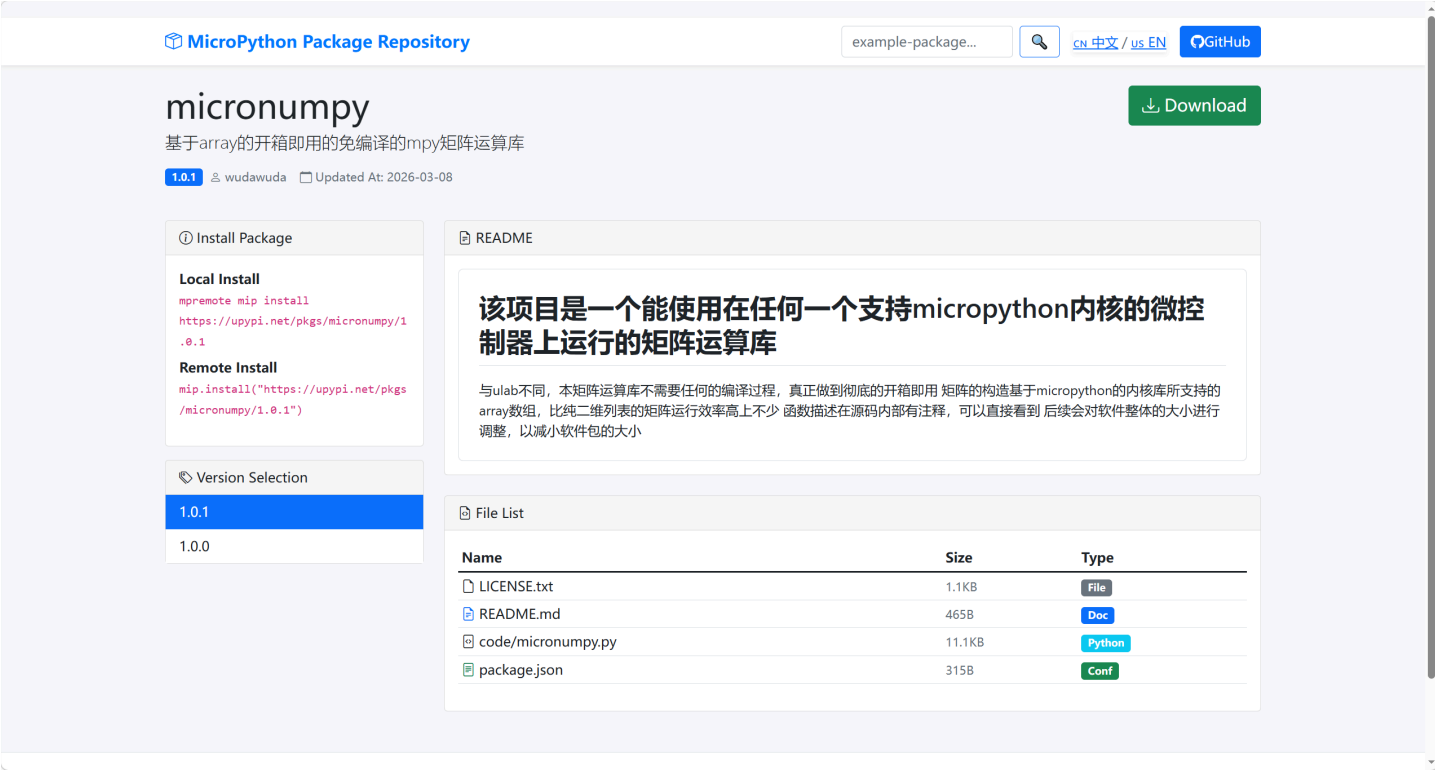
1.1 文档目的

本文档定义了 **Thonny-uPyPi 包管理插件** 的功能需求、非功能需求与开发规范，用于指导插件的设计、开发与验收，实现 Thonny 与 uPyPi（MicroPython 包仓库）的无缝集成。

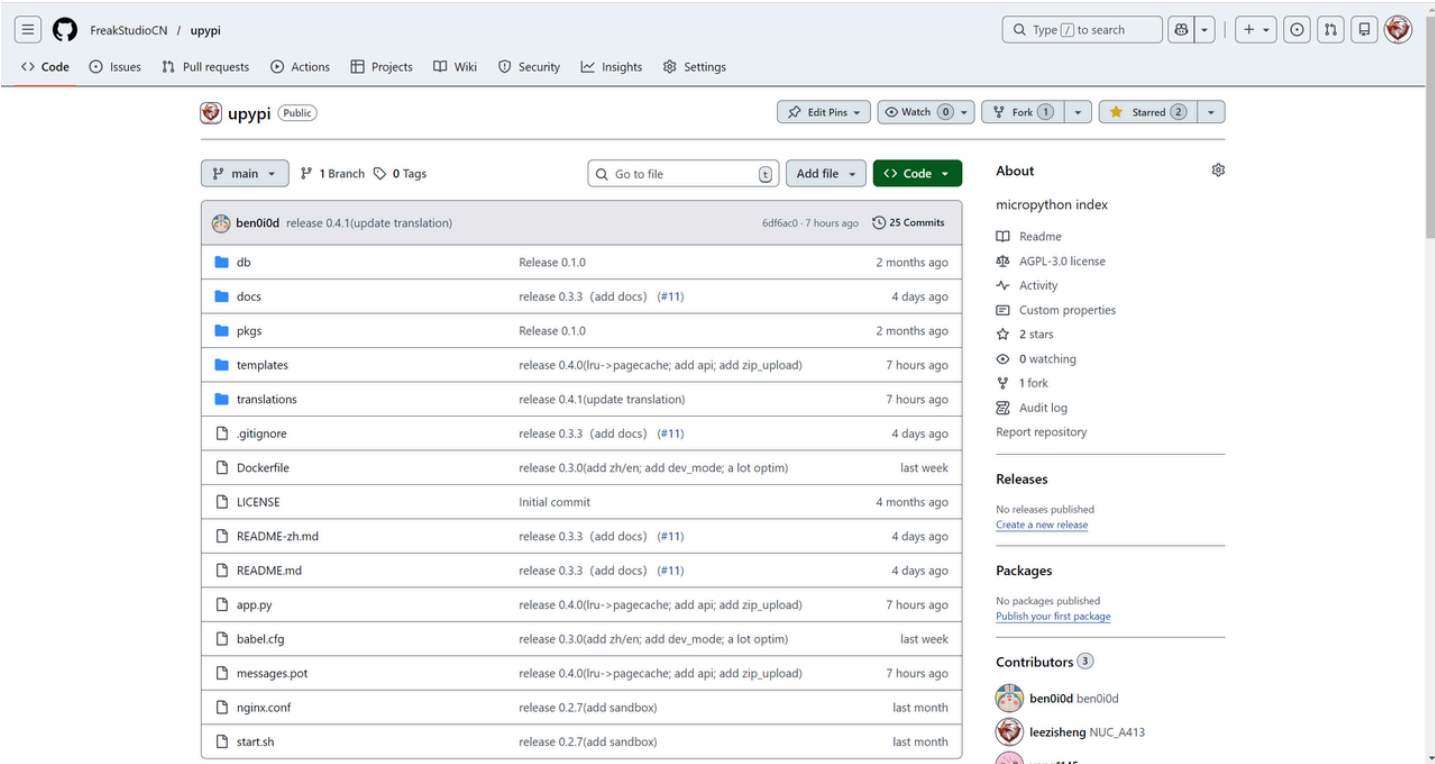
1.2 背景与目标

uPyPi 是类 PyPI 的 MicroPython 包仓库，提供包上传、搜索、下载与分发能力：<https://upypi.net/>





源代码地址：<https://github.com/FreakStudioCN/upypi>



需要开发 Thonny 第三方插件，让开发者在 Thonny 内即可搜索、浏览 uPyPi 包，并一键通过 `mpremote` 安装到开发板，提升 MicroPython 开发效率：

- 适用于 Thonny 较新版本
- 适用于所有支持 MicroPython 的开发板（通过 `mpremote` 连接）
- 遵循 Thonny 插件开发规范（`thonnycontrib` 命名空间、`load_plugin` 入口函数）

二、核心功能需求

2.1 插件入口与界面

- **入口展示：**在 Thonny 主界面新增「uPyPi 包管理」菜单 / 面板入口（可在「工具」菜单或侧边栏展示），支持打开 / 关闭插件面板。
- **界面布局：**
 - 顶部：搜索框 + 搜索按钮
 - 中部：搜索结果列表（支持分页 / 滚动）
 - 底部：安装日志 / 状态提示区

2.2 包搜索功能

1. **搜索触发：**用户输入包名关键词，点击搜索按钮或按回车键，发起搜索请求。

2. **API 调用：**

- 调用 uPyPi 搜索接口：GET `https://upypi.net/api/search?q={关键词}`
- 接口响应格式：JSON，包含 `query`（搜索关键词）和 `results`（结果数组），每个结果项包含 `name`（包名）和 `url`（包详情地址，格式为 `https://upypi.net/pkgs/{包名}/{版本号}/`）
- 支持**模糊搜索**：接口会自动匹配包含关键词的包名，返回所有相关结果

3. **结果展示：**

- 核心信息展示：
 - 包名：直接从搜索结果的 `name` 字段获取
 - 最新版本号、作者、简短描述：通过包详情接口（拼接 `url` + `package.json`）获取并展示
 - 上传时间：若 uPyPi 接口 / 包详情中提供该字段则展示，否则显示“暂无”
- 状态提示：
 - 空结果提示：当 `results` 数组为空时，显示「未找到匹配的包，请尝试其他关键词」
 - 搜索错误提示：当接口请求失败（如网络错误、服务端异常）时，显示「搜索失败，请检查网络后重试」
 - 加载中提示：搜索请求发起后，显示加载动画或「正在搜索…」提示，提升交互体验

2.3 包下载与安装功能

- **下载操作：**

- 点击「下载」按钮，获取 uPyPi 包的下载链接（如 `https://upypi.net/pkgs/{包名}/{版本号}/`）
- 下载包文件（`.py` 或 `.mpy` 格式）到本地临时目录或 Thonny 项目目录
- **安装操作：**
 - 点击「安装到开发板」按钮，调用 `mpremote` 命令将包文件上传到开发板的 `/lib` 目录
 - 命令示例：`mpremote cp {本地包路径} :/lib/{包名}.py`
- **状态反馈：**
 - 实时显示下载 / 安装进度
 - 安装成功 / 失败提示（弹窗或底部日志区）
 - 失败时展示具体错误信息（如 `mpremote` 未安装、开发板未连接）

三、接口与依赖设计

3.1 uPyPi API 依赖

- 搜索接口：`GET https://upypi.net/api/search?q={query}`
 - 响应格式：JSON，包含 `name`、`url` 字段

```
(mp_env) (base) leeqingshui@leeqingshui-VMware-Virtual-Platform:~/MicroPython0S$ curl https://upypi.net/api/search?q=ds18b20
{"query":"ds18b20","results":[{"name":"ds18b20_driver","url":"https://upypi.net/pkgs/ds18b20_driver/1.0.0"}]}
(mp_env) (base) leeqingshui@leeqingshui-VMware-Virtual-Platform:~/MicroPython0S$ curl https://upypi.net/api/search?q=ds1
{"query":"ds1","results":[{"name":"ads1115_driver","url":"https://upypi.net/pkgs/ads1115_driver/1.0.0"}, {"name":"ds1232_driver","url":"https://upypi.net/pkgs/ds1232_driver/1.0.0"}, {"name":"ds1302_driver","url":"https://upypi.net/pkgs/ds1302_driver/1.0.0"}, {"name":"ds1307_driver","url":"https://upypi.net/pkgs/ds1307_driver/1.0.0"}, {"name":"ds18b20_driver","url":"https://upypi.net/pkgs/ds18b20_driver/1.0.0"}]}
```

代码块

```
1 (mp_env) (base) leeqingshui@leeqingshui-VMware-Virtual-Platform:~/MicroPython0S$ curl https://upypi.net/api/search?q=ds18b20
2 {"query":"ds18b20","results":
3 [{"name":"ds18b20_driver","url":"https://upypi.net/pkgs/ds18b20_driver/1.0.0"}]}
4
5 (mp_env) (base) leeqingshui@leeqingshui-VMware-Virtual-Platform:~/MicroPython0S$ curl https://upypi.net/api/search?q=ds1
6 {"query":"ds1","results":
7 [{"name":"ads1115_driver","url":"https://upypi.net/pkgs/ads1115_driver/1.0.0"},
8 {"name":"ds1232_driver","url":"https://upypi.net/pkgs/ds1232_driver/1.0.0"},
9 {"name":"ds1302_driver","url":"https://upypi.net/pkgs/ds1302_driver/1.0.0"},
10 {"name":"ds1307_driver","url":"https://upypi.net/pkgs/ds1307_driver/1.0.0"},
11 {"name":"ds18b20_driver","url":"https://upypi.net/pkgs/ds18b20_driver/1.0.0"}]}
```

- 包详情接口：`GET https://upypi.net/pkgs/{包名}/{版本号}/package.json`

- 响应格式：直接返回该版本包的 `package.json` 文件，JSON 结构包含以下核心字段：

字段	类型	说明
name	string	包名（如 ds18b20_driver）
version	string	版本号（如 1.0.0）
description	string	包的功能描述
author	string	作者名称
license	string	开源许可证类型（如 MIT）
chips	string	支持的芯片型号，all 表示无限制
fw	string	依赖的固件类型，all 表示无特定固件依赖
_comments	object	对 chips 和 fw 字段的中文注释说明
urls	array[array[string]]	文件列表：二维数组，每个子数组格式为 ["目标文件名", "uPyPi源文件路径"]，表示需要下载的文件
deps	array	（可选）依赖包声明，格式同 mip 标准，用于自动安装依赖

```
(mp_env) (base) leeqingshui@leeqingshui-VMware-Virtual-Platform:~/MicroPythonOS$ curl https://upypi.net/pkgs/ds18b20_driver/1.0.0/package.json
{
  "name": "ds18b20_driver",
  "version": "1.0.0",
  "description": "A MicroPython library to control DS18B20 driver",
  "author": "leeqingshui",
  "license": "MIT",
  "chips": "all",
  "fw": "all",
  "_comments": {
    "chips": "该包支持运行的芯片型号，all表示无芯片限制",
    "fw": "该包依赖的特定固件如ulab、lvgl,all表示无固件依赖"
  },
  "urls": [
    [
      "ds18x20.py",
      "code/ds18x20.py"
    ],
    [
      "onewire.py",
      "code/onewire.py"
    ]
  ]
}
```

代码块

```
1 (mp_env) (base) leeqingshui@leeqingshui-VMware-Virtual-Platform:~/MicroPythonOS$ curl https://upypi.net/pkgs/ds18b20_driver/1.0.0/package.json
2 {
3   "name": "ds18b20_driver",
4   "version": "1.0.0",
5   "description": "A MicroPython library to control DS18B20 driver",
6   "author": "leeqingshui",
7   "license": "MIT",
8   "chips": "all",
9   "fw": "all",
10  "_comments": {
11    "chips": "该包支持运行的芯片型号，all表示无芯片限制",
```

```
12     "fw": "该包依赖的特定固件如ulab、lvgl,all表示无固件依赖"
13 },
14     "urls": [
15         [
16             "ds18x20.py",
17             "code/ds18x20.py"
18         ],
19         [
20             "onewire.py",
21             "code/onewire.py"
22         ]
23     ]
24 }
```

3.2 Thonny 插件 API 依赖

- 入口函数：`load_plugin()`，用于注册插件到 Thonny
- 工作台接口：`get_workbench()`，用于创建面板、添加菜单
- 进程调用接口：用于执行 `mpremote` 命令并获取输出

3.3 外部工具依赖

- `mpremote`：必须已安装并配置到系统 PATH，用于开发板通信
- Python 3.8+：Thonny 内置 Python 环境需满足版本要求

四、开发规范

- 包名：`thonny-upypi-manager`（以 `thonny-` 开头，符合 Thonny 插件命名规范）
- 打包格式：`wheel`（`.whl`），便于上传到 PyPI
- `setup.py` / `pyproject.toml` 中声明依赖：`mpremote`、`requests`（用于 API 调用）
- 声明支持的 Thonny 版本范围：`>=3.0,<5.0`

五、验收标准

- 功能验收：
 - 插件能正常在 Thonny 中加载，入口可见
 - 搜索功能能正确返回 uPyPi 上的包信息
 - 能正常下载包并通过 `mpremote` 安装到开发板
 - 错误场景（无网络、开发板未连接）能正确提示

- 兼容性验收：
 - 在 Windows/macOS/Linux 下均能正常运行

版本记录

版本号	修改人员	时间	内容
v1.0	李子圣	2026/03/13	初始化文档。
v1.1	李子圣	2026/03/13	修改2.2包搜索功能。

相关参考

thonny / thonny

Google Translate

<> Code Issues 822 Pull requests 19 Discussions Actions Wiki Security Insights

Plugins

Aivar Annamaa edited this page on Mar 11, 2021 · 33 revisions

Plug-ins

NB! The plug-in system was changed in version 3.0

Thonny can be extended with 3rd party plug-ins. Usually they are distributed via <https://pypi.org> and can be installed with Tools => Mangage plug-ins.

About writing plug-ins

A 3rd-party Thonny plug-in is a Python module (or package) located under `thonnycontrib` (or `thonnycontrib.backend`) namespace package and containing the parameterless function `load_plugin`.

Here's an example of a plug-in which adds a command into the "Tools" menu:

```
from thonny import get_workbench
from tkinter.messagebox import showinfo

def hello():
    showinfo("Hello!", "Thonny rules!")

def load_plugin():
    get_workbench().add_command(command_id="hello",
                                menu_name="tools",
                                command_label="Hello!",
                                handler=hello)
```

During startup Thonny locates all such modules and calls their `load_plugin` function.

Setting up the development

Pages 24

Find a page or section...

Home

Assistant

ClassroomSetup

Custom layout

Custom shortcuts

DeploymentOptions

FAQ

Friendly traceback

InstallingPackages

InventorPrime

Linux

MacOSX

MicroPython

Plugins

Plug-ins

```
def load_plugin():
    get_workbench().add_command(command_id="hello",
                                menu_name="tools",
                                command_label="Hello!",
                                handler=hello)
```

Google Translate

MacOSX

MicroPython

Plugins

Plug-ins

About writing plug-ins

Setting up the development

An example setup (with Linux commands)

What can a plug-in do?

Back-end plug-ins

How to package and distribute your plug-in?

Some 3rd party plug-ins

PortableVersion

Show 9 more pages...

Clone this wiki locally

<https://github.com/thonny/thonny.wiki>

During startup Thonny locates all such modules and calls their `load_plugin` function.

Setting up the development

Where does the aforementioned `thonnycontrib` package live? As it is a [namespace package](#), it can be spread out to several places on the Python path. This means you can either use a directory which is already on the Python path or you can create your project anywhere and run Thonny with the project directory added to the path (eg. via `PYTHONPATH` environment variable).

If you don't want to mess with the path then it's easiest to develop and test the plug-in under your Python's user site directory (the exact path is given in the Tools => Manage Plug-ins dialog)

An example setup (with Linux commands)

First make sure that you have installed all the [required Python packages](#) (either with pip into a virtual environment or via system tools).

The following commands show how to set up both Thonny and your plugin project with the hello plugin shown above. When you run thonny on the last line, you should see the new item in the Tools menu.

```
#!bash
cd ~
mkdir thonny_stuff
cd thonny_stuff
git clone https://github.com/thonny/thonny
mkdir -p my_plugin/thonnycontrib
cp hello.py my_plugin/thonnycontrib
export PYTHONPATH=~/.thonny_stuff/my_plugin
cd thonny
python3 -m thonny
```

python3 -m thonny

Google Translate

What can a plug-in do?

In short: whatever.

More pragmatically: `load_plugin` usually retrieves the singleton instance of the class `thonny.workbench.Workbench` (using `thonny.get_workbench()`) and calls its public methods like `add_command` or `add_syntax_theme` with suitable arguments.

At the moment these methods are not documented, so you need to read the code in `thonny.workbench` and/or some existing plug-ins (either 3rd party plug-ins listed above or built-in plug-ins under the `thonny.plugins` package)

Back-end plug-ins

Regular plug-ins work in Thonny's front-end process, but it's also possible to add plug-ins for the CPython back-end. These plug-ins must live under `thonnycontrib.backend`.

TODO: See [___](#) for an example.

How to package and distribute your plug-in?

In short: the same way as you would package and distribute any Python module or package. Just make sure your distribution name starts with `thonny-` and you have specified supported Thonny version range under dependencies.

Note that Thonny uses PyPI's JSON API to get information about your plug-ins dependencies (for an example search `dist_requires` from <https://pypi.org/pypi/thonny-edison/json>). PyPI can extract this information only from whl files and these must be uploaded before other files (eg. tar.gz and zip).

Here is an example: <https://github.com/thonny/thonny-edison>

Some 3rd party plug-ins

See <https://thonny.org/#plugins>

Thonny's main developers are not native English speakers. Feel free to create an issue if you spot a grammar or style error in the wiki.

<https://thonny.org/#plugins>

<https://github.com/thonny/thonny/wiki/Plugins>

<https://github.com/FreakStudioCN/upypi>